

Lesson 10 Teacher's Script

Core Mechanics IV: Play It Again

VIDEO GAME CREATION

[0:00-0:05] Welcome Back

"Welcome back! Last week your game finally got a real scoreboard. Quick show of hands - who styled their HUD, or added scoring variety?" [Pause, call on 1-2 volunteers]

"Today we fix something that's probably bugged a few of you since Lesson 8: your game 'ends,' but it doesn't actually stop. Today it finally does - and you can play again without touching the Run button."

[0:05-0:20] The Last Missing Piece: What 'Over' Should Actually Mean

"Right now, when you win or lose, a message shows up and... nothing else changes. Enemies keep coming. You can keep shooting. The numbers might even keep moving. 'Game Over' hasn't actually meant anything yet."

"Today, 'over' means something real: the round stops mattering, and there's a clear way to start a fresh one. That's two pieces - freezing, and restarting - and they both lean on the exact same trick."

[Discussion question] "Think of any game you've played - what usually happens the instant you lose? What are all the different things that might stop or change?" (Draw out: enemies stop, controls lock, a menu or prompt appears, some kind of restart option shows up - list a few examples on the board if helpful.)

[Timing checkpoint: wrap by 0:20]

[0:20-0:40] One Setup, Two Doors In

"Here's the trick: your game already HAS code that sets everything up - score at 0, lives at 3, enemies in place. It's been running since Lesson 5, just once, right at the start. Today, we give that exact same code a second way to run: on demand, whenever R is pressed."

[Demo cue: switch to Godot, open a project with a working scoreboard from Lesson 9]

"Watch: I'm taking the setup - score at 0, lives at 3, enemies in place - and moving it into its own function on my GameManager, start_game(). The enemy-spawning loop that used to run in _ready() becomes spawn_enemies() on the Main scene, and start_game() calls it. Then GameManager's _ready() just calls start_game once, deferred - Godot runs a child's _ready() before its parent has finished loading, and the parent won't accept new children until it has, so call_deferred waits for that. That's the whole refactor." [Build it live] "Now here's the payoff: I can

call start_game() again, any time I want, from anywhere else in my code - like, say, when R is pressed." [Wire up the restart key, demonstrate a full reset]

"Notice nothing about the SETUP LOGIC itself changed. We didn't write a second version of it for restarting. We just gave the existing version a second doorway in."

[Discussion question] "Why is it better to have ONE piece of setup code with two ways to trigger it, instead of writing the setup code twice - once for the start, once for restart?" (Draw out: if you ever need to change how the game sets up - say, starting with 4 lives instead of 3 - you'd only have to change it in one place instead of two, and remembering to update both copies is exactly the kind of thing that's easy to forget.)

[Timing checkpoint: wrap by 0:40]

[0:40-0:55] The Freeze: A Flag That Gates Everything

"Restarting is only half of today. The other half: actually STOPPING the game once it's over, so nothing keeps happening behind that win/lose message."

[Demo cue: same Godot project] "I've added one new variable on GameManager - game_active, true by default. The moment you win or lose, it flips to false. And now, at the very top of my enemy's movement code, one line: if the game isn't active, don't do anything else this frame." [Show enemies freezing on game-over]

"One flag, checked in the right place, freezes the whole game. That's genuinely it - no special 'pause' feature required, just a question asked at the right moment, every frame: 'is the game still going?"'

[Show the next slide: the Activity Diagram] "Here's the whole round in one picture. The round ends, game_active flips to false, and everything freezes. Then we ask two questions when R is pressed: was R actually pressed, and is the round actually over? Only if both answers are yes do we run start_game() - the same one reset routine that runs when the game first launches. Press R in the middle of a round and the answer to the second question is no, so nothing happens."

[Show the next slide: What Is Game State?] "One more idea before lab, because it will save you an hour of confusion. Everything your game remembers right now is called its game state: the score, the lives, how many enemies are left, whether the round is over, where the player is standing, which picture the player is showing, even whether a sound or an animation is mid-play. Until today, your game only ever started once and ended once, so nothing had to be put back. A restart changes the rule: every single piece of state has to go back to how it was at the start."

[Show the next slide: The Restart Audit] "So today you audit. Find every place your game does something when it starts - in Scratch, every 'when green flag clicked' script on every sprite and the stage; in Godot, every _ready(); in GameMaker, every Create event; in Unity, every Awake and Start. Then find everything you did at game over - an animation, a color change, a hidden sprite, a sound, a message. If something changed when the round ended, something has to change it back when the round restarts. If you added something extra that isn't in the handout, it is on your list too, because the handout can't know about it."

[Discussion question] "What do you think would happen if we forgot to check game_active somewhere - say, in the code that handles a bullet hitting an enemy? " (Draw out: that specific thing would keep happening even after game-over - a good preview of exactly the kind of bug students should test for today.)

[Timing checkpoint: wrap by 0:55]

[0:55-1:00] Today's Exercise & Handouts

"For lab today, open your own project from last week - not a new one, and no new assets needed. Every engine's handout has the same two moves: turn your existing setup code into something callable again, and add a game_active-style flag that freezes things once the round ends."

[Hand out engine-specific handouts]

"Test in both directions today, and in both game-end states: play to a win, restart, confirm it's clean. Then play to a loss, restart, confirm that's clean too. And don't skip testing that R does NOTHING while a round is still active - that's an easy thing to accidentally break."

"Questions before we start lab? ... Let's make your game actually replayable."

Lab Time Notes

- The single most important thing to verify this week: a full round-trip, twice - win then restart, and separately lose then restart. Both should leave the game in an identical, clean starting state.
- Most likely mistake across every engine: forgetting to clear old enemies before respawning on restart, leading to doubled enemies or a wrong count after the first restart. Check for this explicitly, don't just eyeball the screen.
- Game state drift: any start-up code or game-over effect a student added beyond the handout (extra green-flag scripts, animations, color or costume changes, hidden or moved objects, sounds) is NOT reset by the handout steps. Walk the room with the Restart Audit checklist from the handout; the symptom is usually a restart that looks almost right (a player stuck in the wrong costume, a message still showing, an animation that never replays).
- Second most likely mistake: R doing something mid-game because the game_active check was left out of the restart-key logic. Test this directly by pressing R while actively playing.
- This is a genuinely satisfying lesson to finish - encourage a full playthrough as part of the end-of-lesson share-out (win, restart, lose, restart) since it's the first time the game truly feels replayable, not just winnable once.
- Level 3 bug hunt: there is a real bug in the win/lose logic from Lessons 6 and 8. A round with some kills and some escapes (say 3 and 2) ends with no win and no loss, and R does nothing. Ask early finishers to find it by playing a mixed round; accept any fix that guarantees every round ends in exactly one outcome. The shown fix: an escape also reduces enemies_remaining, and the win check requires lives above 0.
- Students who finish Level 2 early should be pointed at the Level 3 ideas on their handout, not given new unrelated work.